1. Current Efforts of Typing the Nix Language and Coding an LSP

In this document I try to lay out the current efforts of creating a type system and its implementation as a language server written in Rust. This document should act as an overview such that we (Peter Thiemann, Taro Sekiyama and I) have a common ground to discuss next steps and current efforts. Most of it is WIP and there are many loose ends and even contradictions.

2. Syntax

2.1. Syntax: Basetypes

Definition of some base types using regular expressions.

$$
\begin{aligned}
c &::= [\hat{}"\$\backslash] \mid \$(?!\{) \mid \backslash. \\
\text{inter} &::= \$\{\hat{}\}*\} \\
\textbf{String}\ s &::= "(c* \ \text{inter})* \ c*" \\
\textbf{Boolean}\ b &::= \text{true} \mid \text{false} \\
\textbf{Path}\ p &::= (./|\sim/|/)([\text{a-z A-Z .}]+/?)+ \\
\textbf{Number}\ n &::= ([0\text{-}9]*\backslash.)?[0\text{-}9]+ \\
\textbf{Label}\ l &::= [\text{A-Za-z\_}][\text{A-Za-z0-9\_'-}]* \\
\textbf{Variable}\ v &::= [\text{A-Za-z\_}][\text{A-Za-z0-9\_'-}]*
\end{aligned}
$$

The language consists of the standard base types string, boolean, number and label. Labels are distinct here, because we need a syntactic class in some places where only labels are allowed. An example is a path that is constructed from labels interspersed by dots, i.e. `hm.packages.git`.

2.2. Syntax

$$
\begin{aligned}
t ::= &\mid b \mid s \mid p \mid n \mid l \mid v \mid \text{null} \\
\textbf{Record} &\mid \{\mathrm{a}_i\} \mid \textbf{rec}\ \{\mathrm{a}_i\} \\
\textbf{Array} &\mid [\ t_0\ t_1\ \ldots\ t_n\ ] \\
\textbf{Has-Attribute} &\mid t\ \ ?\ \ l \\
\textbf{Has-Attribute-Or} &\mid t.l\ \textbf{or}\ t \\
\textbf{Record-Concat} &\mid t\ //\ t \\
\textbf{Array-Concat} &\mid t +\!\!+ t \\
\textbf{Lookup} &\mid t.l \\
\textbf{Dynamic-Lookup} &\mid t.t \\
\textbf{Function} &\mid \mathrm{pat} : t \\
&\mid \textbf{let}\ \mathrm{a}_i\ \textbf{in}\ t \\
&\mid \textbf{if}\ t\ \textbf{then}\ t\ \textbf{else}\ t \\
&\mid \textbf{with}\ t; t \\
&\mid \textbf{assert}\ t; t
\end{aligned}
$$

- Functions take one argument which can be a *pattern*. This pattern has a *record-like* structure, allowing multiple fields to be present and also *default arguments*. This way a function taking multiple arguments can be created without resorting to currying.
- Records can be marked *recursive* with the `rec` keyword. Records and Arrays are *immutable* but there are concat operations (**Record-Concat** and **Array-Concat**) that can be used to create new, bigger datatypes.
- Record lookups can be static (with a given label) or dynamic, with an arbitrary expression t, that has to reduce to a string. This is further discussed in Section 5.3.
- Let statements can have multiple bindings $a_1 = t_1; \ldots; a_n = t_n$ before the `in` keyword appears.
- The *with statement* expects an arbitrary expression that reduces to a record. Every field from the record is then added to the scope of the next expression without shadowing existing variables. This is further discussed in Section 6.1.

2.3. Syntax: Record and Let fields

$$
\begin{aligned}
\text{a} &::= l = t; \mid \text{i} \\
\text{i} &::= \textbf{inherit}\ l_0\ \ldots\ l_n; \mid \textbf{inherit}\ (p)\ l_0\ \ldots\ l_n; \\
p &::= l \mid p.l
\end{aligned}
$$

Both let-statements and records allow *inherit statements* to be placed between ordinary field declarations. Inherit statements take a known label for a value and *reintroduce* the label as "label = value;" to the record or let expression. This feature is only syntactic sugar to make it easier to build records.

Let statements can take a root path `(p)` which is prefixed to all following lookups. This way, a deep record can be referenced from which all values are taken. For example, the statement `inherit (world.objects.players) robert anders;` will add `robert = world.objects.players.robert; anders = world.objects.players.anders;` to the surrounding record or let expression.

2.4. Syntax: Pattern

$$
\begin{aligned}
\text{pat} &::= \{\ e_i\ \} \\
&\mid \{\ e_i, \ldots\ \} \\
&\mid l \\
\text{e} &::= l \mid l\ ?\ t
\end{aligned}
$$

Patterns can be open (...) or closed and can also be given default arguments with the `?` syntax. An example would be `{a, b ? "pratt", …}` which is an *open* pattern with a default value of "pratt" for the label b.

2.5. Evaluation Contexts

$$E ::= [] \mid Ee \mid (E).l \mid \text{if}\ \ E\ \ \text{then}\ \ t\ \ \text{else}\ t \mid E + t \mid v + E$$

2.6. Reduction Rules

Let t, t_1 and t_2 range over syntax terms and l, v over identifiers (labels and variables). H stores and memoizes thunks that are used during evaluation and sometimes left out if unchanged.

$$
\begin{aligned}
\langle (l : t_2)t_1, H\rangle &\longrightarrow \langle t_2[l := a], H[a = t_1]\rangle && \text{R-Fun} \\
\langle (\{l_i\} : t)\{l_i = t_i;\}, H\rangle &\longrightarrow \langle t[l_i := a_i], H[a_i = t_i]\rangle && \text{R-Fun-Pat} \\
\langle (\{l_i, ...\} : t)\{l_i = t_i; ..\}, H\rangle &\longrightarrow \langle t[l_i := a_i], H[a_i = t_i]\rangle && \text{R-Fun-Pat-Open} \\
\langle (\{l_i\ ?\ t_i, l_j\} : t_2)(\{l_k = t_k; l_j = t_j;\}), H\rangle &\longrightarrow \langle t_2[l_m := a_m][l_n = a_n][l_j := a_j], H[..]\rangle && \text{R-Fun-Pat-Default} \\
&\text{Where } m = \{i : \exists k.i = k\}; n = \{i : \nexists k.i = k\} && \\
\langle \{l : t, ..\}.l, H\rangle &\longrightarrow \langle \text{fv}(t), H\rangle && \text{R-Lookup} \\
\langle (\{l_i = t_i\}).l, H\rangle &\longrightarrow \langle \text{null}, H\rangle \text{ if } \nexists j.l_j = l && \text{R-Lookup-Null} \\
\langle (l : t_1, \{..\}).l \text{ or } t_2, H\rangle &\longrightarrow \langle \text{fv}(H[t_1]), H\rangle && \text{R-Lookup-Default-Pos} \\
\langle (\{..\} \setminus l).l \text{ or } t, H\rangle &\longrightarrow \langle t, H\rangle && \text{R-Lookup-Default-Neg} \\
\langle (\{..\} \setminus l)\ ?\ l, H\rangle &\longrightarrow \text{false} && \text{R-Has-Pos} \\
\langle \{l : t, ..\}\ ?\ l, H\rangle &\longrightarrow \text{true} && \text{R-Has-Neg} \\
\langle \text{let } l_i = t_i; \text{in } t_2, H\rangle &\longrightarrow \langle t_2[l_i = v_i], H[v_i = t_i]\rangle && \text{R-Let} \\
\langle \text{with } \{l_i = t_i\}; t_2, H\rangle &\longrightarrow \langle t_2[l_i \ominus\ a_i], H[a_i = t_i]\rangle && \text{R-With} \\
\langle \text{if true then } t_1 \text{ else } t_2, H\rangle &\longrightarrow \langle t_1, H\rangle && \text{R-Cond-True} \\
\langle \text{if false then } t_1 \text{ else } t_2, H\rangle &\longrightarrow \langle t_2, H\rangle && \text{R-Cond-False} \\
\langle t_1 +\!\!+ t_2, H\rangle &\longrightarrow \langle [...\text{fv}(H(t_1)), ...\text{fv}(H(t_2))], H\rangle && \text{R-Array-Concat} \\
\langle t_1\ \ //\ t_2, H\rangle &\longrightarrow \langle \{...t_2, ...t_1\}, H\rangle && \text{R-Record-Concat}
\end{aligned}
$$

- fv (=force value) is a function that computes the thunk on first request, or returns the cached value on consecuetive requests.

2.7. Evaluation Rules

$$\textsc{F-Force-Step}\quad \frac{\langle a, H[a \to e]\rangle \quad \langle e, H\rangle \to \langle e', H'\rangle}{\langle a, H\rangle \to \langle a, H'[a \to e']\rangle} \qquad \textsc{F-Force-Value}\quad \frac{H[a \to v] \quad v : \text{Value}}{\langle a, H\rangle \to \langle v, H\rangle}$$

The *spread syntax* {...rc} means a new record is created from the fields of `rc` where `rc` is a record. These new fields never overwrite existing fields, meaning `{a : int, …{a : string}}` will reduce to `{a: int}`. Similar is possible for arrays, but naturally without deduplication. For two records $A : \{l_i : t_i\}$ and $B : \{l_i : t_i\}$ this means $\{..A, ..B\} = \{l_a = t_a; l_b = t_b;\}$ where $a \in \{i : l_i \in A\}$ and $b \in \{i : l_i \in (B \setminus A)\}$. $B \setminus A$ is the Record B where every label i has been removed if it is in A.

I also oftentimes abbreviate $l_0 ... l_n$ as l_i. Two dots (..) denote that there are other bindings possible in a record where as three dots (...) are used for spread syntax and the open pattern.

- R-Fun is the standard β-reduction for functions where the argument is replaced by the supplied argument's value in the body. `b[l := a]` means that the variable l is assigned value a in the body.
- I use the syntax `{..} \ l` to create an arbitrary record without the label l.
- To reduce with statements the first term has to reduce to a record and I don't like the formalization of that currently. For the next expression the record fields are added to the scope without shadowing existing bindings. I use the `/=` operator to get this behavior. See Section 6.1 for further discussion.

3. Type System

What follows are the typing and subtyping rules as well as an overview over the constraint subroutine.

3.1. Values

$$p : t \quad | \quad x; \quad | \quad \{..\} \quad | \quad \text{rec } \{..\}$$

3.2. Types

$$\begin{aligned}
\text{Booleans } b &:= \text{true} \mid \text{false} & \text{Type } \tau &::= \tau \to \tau \mid \alpha \mid \top \mid \bot \\
\textbf{Type Connectives} &\mid \tau \sqcup \tau \mid \tau \sqcap \tau \\
\textbf{Recursion} &\mid \mu\alpha\ \tau \\
\textbf{Base Types} &\mid \text{bool} \mid \text{string} \mid \text{path} \mid \text{num} \\
\textbf{Records} &\mid \{l_0 : \tau \ ... \ l_n : \tau\} \mid \langle l_0 : \tau \ \ ... \ \ l_n : \tau\rangle \\
\textbf{Lists} &\mid [\ \tau\ ] \mid [\ \tau_1 \ ... \ \tau_n\ ] \\
\textbf{Patterns} &\mid (\{l_0 : p; ...; l_n : p\}, b) \\
\text{Pattern Elemenst } p &:= \tau \mid \tau^?
\end{aligned}$$

- TODO: Do we need an option type because we have functions with default arguments?
  - From pattern to var?
- TODO: Pattern Type should be as expressive as the syntax construct

T-Var

$$\frac{x : \forall \vec{\alpha}.\ \tau \in \Gamma}{\Gamma \vdash x : \tau[\vec{\alpha} \setminus \vec{\tau}]}$$

T-App

$$\frac{\Gamma \vdash t_1 : \tau_1 \to \tau_2 \quad \Gamma \vdash t_2 : \tau_1}{t_1 t_2 : \tau_2}$$

T-Abs

$$\frac{\Gamma, x : \tau_1 \vdash t : \tau_2}{\Gamma \vdash (x : t) : \tau_1 \to \tau_2}$$

T-Abs-Pat

$$\frac{\Gamma, \overline{x : \tau}^i \vdash t : \tau_2}{\Gamma \vdash (\{\overline{x}^i\} : t) : \overline{\tau}^i \to \tau_2}$$

T-Abs-Pat-Opt

$$\frac{\text{TODO}}{\text{TODO}}$$

T-Rcd

$$\frac{\Gamma \vdash t_0 : \tau_0 \quad \ldots \quad \Gamma \vdash t_n : \tau_n}{\Gamma \vdash \left\{\vec{l} : \vec{t}\right\} : \left\{\vec{l} : \vec{\tau}\right\}}$$

T-Proj

$$\frac{\Gamma \vdash t : \{l : \tau\}}{\Gamma \vdash t.l : \tau}$$

T-Sub

$$\frac{\Gamma \vdash t : \tau_1 \quad \tau_1 \leq \tau_2}{\Gamma \vdash t : \tau_2}$$

T-Negate

$$\frac{\Gamma \vdash e : \text{bool}}{\Gamma \vdash !e : \text{bool}}$$

T-Check

$$\frac{\Gamma \vdash e : \{..\}}{\Gamma \vdash e?\, l : \text{bool}}$$

T-Or-Neg

$$\frac{\Gamma \vdash t_1 : \{l : \tau_1\} \quad \Gamma \vdash t_2 : \tau_2}{\Gamma \vdash (t_1).l \text{ or } t_2 : \tau_1}$$

T-Or-Pos

$$\frac{\Gamma \vdash t_1 : \tau_1 \quad l \notin \tau_1 \quad \Gamma \vdash t_2 : \tau_2}{\Gamma \vdash (t_1).l \text{ or } t_2 : \tau_2}$$

T-Lst-Hom

$$\frac{\Gamma \vdash t_0 : \tau \quad \ldots \quad \Gamma \vdash t_n : \tau}{\Gamma \vdash [\ t_0\ t_1\ \ldots\ t_n\ ] : [\tau]}$$

T-Lst-Agg

$$\frac{\Gamma \vdash t_0 : \tau_0 \quad \ldots \quad \Gamma \vdash t_n : \tau_n \quad \exists i, j . \tau_i \neq \tau_j}{\Gamma \vdash [\ t_0\ t_1\ \ldots\ t_n] : [\tau_0\ \tau_1\ \ldots\ \tau_n]}$$

T-List-Concat-Hom

$$\frac{\Gamma \vdash a : [\tau] \quad \Gamma \vdash b : [\tau]}{\Gamma \vdash a +\!\!+ b : [\tau]}$$

T-List-Concat-Multi

$$\frac{\Gamma \vdash a : [\vec{\tau_1}] \quad \Gamma \vdash b : [\vec{\tau_2}]}{\Gamma \vdash a +\!\!+ b : [\vec{\tau_1}\vec{\tau_2}]}$$

T-Rec-Concat

$$\frac{\Gamma \vdash a : \{l_i : \tau_i\} \quad \Gamma \vdash b : \left\{l_j : \tau_j\right\}}{\Gamma \vdash a \ //\ b : \{..b, ..b\}}$$

T-Multi-Let

$$\frac{\overline{\Gamma[x_i : \tau_i \vdash t_i : \tau_i]}^i \quad \overline{\Gamma[x_i : \forall \vec{\alpha}.\tau_i]}^i \vdash t : \tau}{\Gamma \vdash \text{let } x_0 = t_0; \ldots; x_n = t_n \text{ in } t : \tau}$$

T-If

$$\frac{\Gamma \vdash t_1 : \text{bool} \quad \Gamma \vdash t_2 : \tau \quad \Gamma \vdash t_3 : \tau}{\text{if } t_1 \text{ then } t_2 \text{ else } t_3 : \tau}$$

T-With

$$\frac{\Gamma \vdash t_1 : \left\{\vec{l} : \vec{\tau}\right\} \quad \Gamma, l_0 : \tau_0, \ldots, l_n : \tau_n \vdash t_2 : \tau \quad l_i \notin \Gamma}{\Gamma \vdash \text{with } t_1; t_2 : \tau}$$

T-Assert-Pos

$$\frac{\Gamma \vdash t_1 : b \quad \Gamma \vdash t_2 : \tau_2}{\Gamma \vdash \text{assert } t_1; t_2 : \tau_2}$$

- We have a standard typing context Γ, pre-filled with the standard library functions from Section 10 and functions to handle the basic logic, arithmetic and comparison operators.
- $\forall \vec{a}$ represents a *type scheme* with many polymorphic variables α_i. These are used for let-polymorphism.
- TODO: "T-Rec-Concat" doesn't work really because of the generic subtyping rule. Further discussed in Section 5.1
- TODO: T-multi-let can be made simpler because we can always rewrite multi-let to let-chains. Recursion has to be accounted for, that is still an open question.
- TODO: T-With $l_i \notin \Gamma$ is too restrictive because shadowing labels are allowed, they will just not be used.

3.4. Subtyping Rules

S-Refl
$$\frac{}{\tau \leq \tau}$$

S-Trans
$$\frac{\Sigma \vdash \tau_0 \leq \tau_1 \quad \Sigma \vdash \tau_1 \leq \tau_2}{\Sigma \vdash \tau_0 \leq \tau_2}$$

S-Weaken
$$\frac{H}{\Sigma \vdash H}$$

S-Assume
$$\frac{\Sigma, \triangleright H \vdash H}{\Sigma \vdash H}$$

S-Hyp
$$\frac{H \in \Sigma}{\Sigma \vdash H}$$

S-Rec
$$\frac{}{\mu\alpha.\tau \equiv [\mu\alpha.\tau/\alpha]\tau}$$

S-Or
$$\frac{\forall i, \exists j, \Sigma \vdash \tau_i \leq \tau'_j}{\Sigma \vdash \sqcup_i \tau_i \leq \sqcup_j \tau'_j}$$

S-And
$$\frac{\forall i, \exists j, \Sigma \vdash \tau_j \leq \tau'_i}{\Sigma \vdash \sqcap_j \tau_j \leq \sqcap_i \tau'_i}$$

S-Fun
$$\frac{\triangleleft\, \Sigma \vdash \tau_0 \leq \tau_1 \quad \triangleleft\, \Sigma \vdash \tau_2 \leq \tau_3}{\Sigma \vdash \tau_1 \longrightarrow \tau_2 \leq \tau_0 \longrightarrow \tau_3}$$

S-Rcd
$$\frac{}{\{\vec{t} : \vec{\tau}\} \equiv \sqcap_i \{l_i : t_i\}}$$

S-Depth
$$\frac{\triangleleft\, \Sigma \vdash \tau_1 \leq \tau_2}{\Sigma \vdash \{l : \tau_1\} \leq \{l : \tau_2\}}$$

S-Lst
$$\frac{\Gamma \vdash \tau_1 \leq \tau_2}{\Gamma \vdash [\tau_1] \leq [\tau_2]}$$

- ▷ and ◁ are used to add and remove *typing hypotheses* that are formed during subtyping. Since applying such a hypothesis right after assumption, the later modality ▷ is added and can only be removed after subtyping passed through a function or record construct. **TODO: check**

What follows are the constraining rules used in the constrain subroutine of the implementation. It uses the subtyping rules and applies them to types. The underlying algorithm uses *levels* to distinguish type variables that should be generalized and not. When entering a let-binding, the level is increased as every new type variable should adhere to *let-polymorphism*. During type inference, the algorithm also keeps track of the current level and only generalizes variables that are above the current level. This is done by cloning the inherent structure of the type but adding new type variables. In the rust implementation this is done by the `freshen_above()` function.

3.5. Constraining rules

Constraining takes two types τ_1 and τ_2 and constraints the first type to be subtype of the other.

$$
\begin{array}{rll}
(\tau_1 \to \tau_2), (\tau_3 \to \tau_4) & \rightsquigarrow \text{constrain}(\tau_3, \tau_1); \text{constrain}(\tau_2, \tau_4) & \text{C-Fun} \\
\{\tau_1\}, \{\tau_2\} & \rightsquigarrow \forall i \in \tau_2.\ \text{constrain}(\tau_{1i}, \tau_{2i}) \quad \text{if A} & \text{C-Rec} \\
\{\tau_1\}, (\{\tau_2\}, \textbf{true}) & \rightsquigarrow \forall i \in \tau_2.\ \text{constrain}(\tau_{1i}, \tau_{2i}) \quad \text{if A} & \text{C-Pat-Open} \\
\{\tau_1\}, (\{\tau_2\}, \textbf{false}) & \rightsquigarrow \forall i \in \tau_2.\ \text{constrain}(\tau_{1i}, \tau_{2i}) \quad \text{if A} \wedge \text{B} & \text{C-Pat-Closed} \\
[\tau_1], [\tau_2] & \rightsquigarrow \text{constrain}(\tau_1, \tau_2) & \text{C-Array} \\
(\text{lo}, \text{up})^n, \tau^m \quad \text{if } m \le n & \rightsquigarrow \text{up} \doteq \tau; \forall l \in \text{lo}.\ \text{constrain}(l, \tau) & \text{C-Var-}\star \\
\tau_1^n, \tau_2 & \rightsquigarrow \text{constrain}(\tau_1, \text{ extrude}(\tau_2, \text{ false, n})) & \text{C-Var-}\star \\
\tau^n, (\text{lo}, \text{up})^m \text{ if } n \le m & \rightsquigarrow \text{lo} \doteq \tau; \forall u \in \text{ul}.\ \text{constrain}(\tau, u) & \text{C-}\star\text{-Var} \\
\tau_1, t_2^m & \rightsquigarrow \text{constrain}(\text{extrude}(\tau_1, \text{ true, m}), \tau_2) & \text{C-}\star\text{-Var}
\end{array}
$$

Conditions:

- A: Fields in τ_2 must be present in τ_1
- B: τ_1 must only have the fields in τ_2

Remarks

- $(\text{lo}, \text{up})^n$ is used to match a *type variable* and their lower and upper bounds. The superscript gives the *level* of the variable that is used to handle generalization of variables.
- $\text{lo} \doteq \tau$ is a shorthand for $\text{lo} = \text{lo} + \tau$ and used to extend the list of upper or lower bounds.

- C-Fun is standard function subtyping.
- C-Rec implements width-subtyping of records in the standard manner. It also adds depth-subtyping due to recursion.
- C-Pat-open handles open patterns and has similar semantics to record constraining. The rule C-pat-Closed handles closed patterns with the extra condition that t_1 can not have any additional fields to t_2 which is inforced in condition B.
- Homogenous arrays are constrained as one would expect. Heterogenous arrays with many different field types, are constrained in order.
- What follows are the typvariable constraining rules. These depend on the levels of variables and their bounds (lo, up). C-Var-$\star$ handles the case where the constrained var is ow higher type than the constraining var.
- $\text{extrude}(t)$ is used to create a new type of similar shape to the input but fixed type variables. We need this because lower bounds could refer to variables of higher level than the vars level. Since

4. Equality

Attribute sets and lists are compared recursively, and are therefore fully evaluated.

5. Datatypes

5.1. Records

Records are defined very simply in this type system. The only supported record type is a list of `label: type` mappings which can be added during subtyping. There is no way to reorder them, or remove some. During typing, multiple object constraints are concatenated, so there is a way to add new fields.

Two problems occur with the current implementation. Firstly, we have the `//` operator which implements *open record extension*. Given two records `A: { X: string, Y: int }` and `B: { X: int }` the open record concatenation between the two records `(C = A \\ B)` is `C: {X: int, Y: int}`. This together with the generic subtyping rule T-Sub leaves the type system unsound, because fields can be removed, leaving the record B empty (`T-SUB: B -> {}`). In this case, the type system would predict `A.X` to be of type `string` which is simply wrong after the application.

Since there is no way to remove labels from a record, we don't need lacks predicates! The only thing we need to care about is, how to merge record constraints.

5.2. Context Strings

Context strings and dynamic lookup share the same syntax in that you can insert some arbitrary term `t` into braces with the following syntax `${t}`. For ordinary strings and paths, the value of `t` will be coerced into a string and added literally. From a typing perspective, this is the easy case because inserted values get a constraint of string and that's it. For dynamic lookup it gets trickier though.

5.3. Dynamic Lookup

Context strings allow lookups of the form `a.${t}` where t is allowed to be any expression that ultimately reduces to a string. The reduced string is then used to index the record which a is supposed to be. Since a type system only computes a type and not the actual value, the only possible approach to handle first-class labels is to evaluate nix expressions to some extent. Writing a full evaluator is probably too much, but there could be heuristics for simple evaluation. One approach would be to work backwards from return statements in functions up until it gets too unwieldy. This would also mean implementing the standard library functions like map, readToString etc. One ray of hope is that these were probably already implemented in Tvix.

6. Constructs

6.1. With Statements

With statements allow introducing all bindings of a record into the following expression. For this, the first expression (A) in with A; B has to reduce to a record. If that does not work, typing should raise an error. For explicit records, the following typing is straightforward. Just introduce all fields to the scope without shadowing and continue typechecking B. For the case that A is a type variable, it gets tricky however because of the generic subsumption rule. When A is subtyped like follows $A : \{X : \text{int}\} \to A : \{\}$, then the field X would not be accessible in the function body. The second problem is what I call the *attribution problem*. It occurs when there is a chain of with statements with $A; (\text{with } B;)t$ and A and B are type variables. Now when trying to lookup x in t, it is unclear whether x came from B or A.

6.2. Inherit Statements

Inherit statements can be handled as syntactic sugar.

6.3. Function Patterns

Functions are pure and functional which helps in inferring a proper type. Patterns are given as records, showing which exact fields are wanted "as parameters". The ellipsis `(…)` allSows for arbitrary extra fields, and the `?` question mark syntax for default values. To handle these, all expected record fields need to be present in the function argument so a record constraint with these fields can be added to the argument of the function. If a default value is given for some record fields, a constraint can be made on the arguments as well.

6.4. Dunder Methods

There seem to be some special dunder methods for representations which are handled specially by the evaluator. I have not had the chance to look into it further.

7. Laziness and Recursiveness

Laziness and recursion occur in two language constructs. The first one being *recursive records* and the second one being *let bindings*. To evaluate them, a lazy evaluation scheme is needed which is currently implemented as follows: When typing a let binding or record, the algorithm adds all name bindings to the context up-front. This way, referenced values will not be undefined when looked up, even if their definition was not type checked yet. The typecheck algorithm then starts with some arbitrary first label A which may contain an unchecked expression labeled B. When this undefined label B is found, it is simply used to create upper and lower bounds (constraints). For empty type variables that is fine to do, but when we actually check this B, it will unfold and be constrained with upper and lower bounds. These bounds are missing on the typecheck run of A then. An example would be `let f = a: a + 1; x = f b; b = "hi" in {}` In this case b would be constrained to be a number (because of the application and its implication) but afterwards it will get its "real" type which is string. Currently, the constraint error would be placed at the wrong location (that of the true definition).

```
rec { x = { x = x;};}.x;        # → { x = «repeated»; }
let x = {x = y;}; y = x; in x   # → { x = «repeated»; }
```

Listing 1: Examples of recursive patterns from the nix repl

8. A Note about Implementation

One unique problem of nix is that everything (all 100,000 packages, the operating system, and the standard library) are rooted in a *single file* at https://github.com/NixOS/nixpkgs/blob/master/flake.nix or https://github.com/NixOS/nixpkgs/blob/master/default.nix, depending on whether you use a flake based system or not. To not get lost in the weeds, the nix evaluator heavily relies on the laziness features of the language to not evaluate all of the packages exhaustively. For the ultimate goal of auto-completing nixos options one would have to parse and type this very file with the goal to resolve the module system. This includes the standard library and bootstrapping code for the module system. To even reach it, the type inference algorithm has to support the same kind of laziness the nix evaluator uses to not get lost.

8.1. Practical Type Inference in Face of Huge Syntax

Code inference in the general case is similar to depth-first search, digging down one syntax tree and only returning as soon as all branches have been exhausted. Since nix trees are huge, this approach is not feasible and one has to lean towards a breadth-first search style, which focuses on the currently inferred file and stops when "too far away". To achieve this behavior, the inference algorithm at some point has to decide to stop inference and jump to another unfinished function, remembering at which place it left off. In the nix language, there are two natural places to do so. Laziness of records and let statements gives the natural approach that every newly named binding is a stop-point at which inference only proceeds as far as needed. One heuristic could be to go two more functions down and then return to the let or record to generate at least some approximation of the final type.

The import statement semantics of nix come in very handy at this point. Import statements act just as function calls with the only difference being, that the goto location is defined by path and not by name. Other than that, they can take arguments just as a function, and then try to apply given arguments to the file's expression.

This language design comes in very handy because that way, import statements do not occur at the top of the file where it would need to be decided how to continue typechecking them. They occur right at the location where they are needed, sometimes in let statements or record fields. This way, the laziness of records and let statements could already be enough to get laziness into the language. As for the practical approach, I propose a new marker type which can be set to bindings of a context. This marker type should contain all the information to go back to type inference at a previous location. This probably means cloning the context or restoring it to the previous state – cloning is probably easier. Another approach could be to keep the names undefined and add another mapping between names and reconstruction information somewhere that acts as a fallback.

Some real-world example of import:

```
let
  overrides = {
    builtins = builtins // overrides;
  }
  // import ./lib.nix;
in
scopedImport overrides ./imported.nix
```

8.2. Type Inference in a Language Server Setting

A language server setting adds one more level of complexity. A language server has to handle the communication between client (an editor like vim, emacs, vscode, etc.) and the server itself. It will be notified frequently of code changes and has to adapt to these changes almost immediately to not annoy the user. This is why rust-analyzer and nil, which I take as template for my own efforts, have chosen to use or create *incremental computation* frameworks for the rust language. The one used by rust-analyzer and nil (which is based off of rust-analyzer) is *salsa*. The name stems from the underlying red-green algorithm that decides whether a function needs to be reevaluated because its arguments changed or whether the memoized return value can be returned immediately. In the end, salsa consists of *inputs*, *tracked functions* and *tracked structs*. Inputs are divided into their durability and given to tracked functions. These tracked functions record the inputs and do some arbitrary computation with them. During these computations, the functions might create immutable tracked structs which can act as new inputs to other tracked functions. TraScked structs are interned into a db and act as a single identifier which are cheap to copy around and provide great performance benefits. With these components alone it is possible to create a hierarchy of pure functions that allow for reproducibility.

When implementing this incrementality framework one has to decide where to draw the line between tracking everything too closely such that the framework bloat adds latency and tracking too few intermediate results such that recomputation is heavy again. I currently choose to track inputs, and functions as well as initial calls.

The generalized structure of the three language servers has this structure. A user opens a file and the lsp client sends the text to the language server. The language server stores the text somewhere and adds it to the typing pipeline. The first step of this pipeline is of course lexing and parsing the file. Nil already provides a parser for lossless syntax trees that are handy for error reporting. The file is then lowered into another HIR which is more or less syntax independent and thus changes less frequently. This is necessary because otherwise everything would have to be recomputed all the time. After this, the HIR is given to the inference algorithm that tries to infer a type.

I am currently working to transition from salsa 0.17-pre2 to salsa 0.24 which is the newest version of salsa. As a lot has changed and virtually every part of code is touched, this is very time consuming.

9. Code Overview

Inputs of LSP:

- `File {content: string, }`

Inputs of infer:

- `AST { With(ExprId, ExprId) }` (lowered AST with expressions from the arena)

Tracked structs:

- `Ty { Lambda(Ty), With(Ty, Ty)}` (enum that stores the whole AST)
- `Context {bindings: Vec<_>, }`
- `TyVar {lower_bounds: Vec<Ty>, upper_bounds: Vec<Ty>, level: int}`

Functions:

- `infer` (main work)
  - calls itself with subtrees of the AST and new contexts
  - **Mutates** context
- `constrain` (constrains two types to be the same)
  - calls itself with subtrees of Ty and might cycle
  - **Mutates** Type variables → **Changes context**
- `coalesce` (reduce types to unions and intersections)
  - Create new types
- `extrude` (fix levels of problematic variables in a type scheme)
  - only creates new types
- `freshen_above` (Add new type variables at level > x)
  - only creates new types

10. Appendix A

10.1. List of Builtins

- **abort** `s` : Abort Nix expression evaluation and print the error message `s`.
- **add** `e1 e2` : Return the sum of the numbers `e1` and `e2`.
- **addDrvOutputDependencies** `s` : Copy string `s` while turning constant string context elements into derivation-deep string context.
- **all** `pred list` : Return `true` if `pred` returns `true` for all elements of `list`, else `false`.
- **any** `pred list` : Return `true` if `pred` returns `true` for any element of `list`, else `false`.
- **attrNames** `set` : Return the attribute names of `set`, sorted alphabetically.
- **attrValues** `set` : Return the values of attributes in `set`, ordered by sorted names.
- **baseNameOf** `x` : Return the last component of path or string `x`.
- **bitAnd** `e1 e2` : Bitwise AND of integers `e1` and `e2`.
- **bitOr** `e1 e2` : Bitwise OR of integers `e1` and `e2`.
- **bitXor** `e1 e2` : Bitwise XOR of integers `e1` and `e2`.
- **break** `v` : In debug mode, pause evaluation and enter REPL; otherwise return `v`.
- **builtins** : A set containing all built-in functions and values.
- **catAttrs** `attr list` : Collect the attribute `attr` from each set in `list`, ignoring sets without it.
- **ceil** `double` : Round `double` up to the nearest integer.
- **compareVersions** `s1 s2` : Compare version strings; returns `-1`, `0`, or `1`.
- **concatLists** `lists` : Flatten a list of lists into a single list.
- **concatMap** `f list` : Equivalent to `concatLists (map f list)`.
- **concatStringsSep** `sep list` : Join strings in `list` with separator `sep`.
- **convertHash** `args` : Convert a hash string between formats (base16, sha256, SRI, etc.).
- **currentSystem** : System string like `"x86_64-linux"`.
- **currentTime** : Unix time at moment of evaluation (cached).
- **deepSeq** `e1 e2` : Like `seq`, but fully evaluate nested structures in `e1` first.
- **dirOf** `s` : Directory component of string `s`.
- **div** `e1 e2` : Integer division.
- **elem** `x xs` : `true` if `x` is in list `xs`.
- **elemAt** `xs n` : Return the `n`-th element of `xs`.
- **false** : Boolean literal `false`.
- **fetchClosure** `args` : Fetch a store path closure from a binary cache.
- **fetchGit** `args` : Fetch a Git repo or revision.
- **fetchTarball** `args` : Download and unpack a tarball.
- **fetchTree** `input` : Fetch a tree or file with metadata.
- **fetchurl** `arg` : Download a URL and return store path.
- **filter** `f list` : Return elements where `f` yields `true`.
- **filterSource** `pred path` : Copy sources filtering by `pred`.
- **findFile** `search lookup` : Search `lookup` in `search` path.
- **floor** `double` : Round `double` down to nearest integer.
- **foldl'** `op nul list` : Left fold over `list` with `op`.
- **fromJSON** `e` : Parse JSON string `e` into Nix value.
- **fromTOML** `e` : Parse TOML string `e` into Nix value.
- **functionArgs** `f` : Return formal argument set of function `f`.
- **genList** `generator length` : Generate list of given `length` using `generator`.
- **genericClosure** `attrset` : Compute transitive closure of a relation.
- **getAttr** `s set` : Return attribute `s` from `set`.
- **getContext** `s` : Return derivation context of string `s`.
- **getEnv** `s` : Return environment variable `s`.
- **getFlake** `args` : Fetch flake reference and outputs.
- **groupBy** `f list` : Group elements by key `f(element)`.
- **hasAttr** `s set` : `true` if `set` has attribute `s`.
- **hasContext** `s` : `true` if string `s` has nonempty context.
- **hashFile** `type p` : Compute hash of file at `p`.

- **hashString** `type s` : Compute hash of string `s`.
- **head** `list` : First element of `list`.
- **import** `path` : Load and evaluate Nix file at `path`.
- **intersectAttrs** `e1 e2` : Attributes in `e2` whose names occur in `e1`.
- **isAttrs** `e` : `true` if `e` is an attribute set.
- **isBool** `e` : `true` if `e` is a boolean.
- **isFloat** `e` : `true` if `e` is a float.
- **isFunction** `e` : `true` if `e` is a function.
- **isInt** `e` : `true` if `e` is an integer.
- **isList** `e` : `true` if `e` is a list.
- **isNull** `e` : `true` if `e` is `null`.
- **isPath** `e` : `true` if `e` is a path.
- **isString** `e` : `true` if `e` is a string.
- **langVersion** : Integer of current Nix language version.
- **length** `e` : Length of list `e`.
- **lessThan** `e1 e2` : `true` if `e1 < e2`.
- **listToAttrs** `e` : Convert list of `{name, value}` to attrset.
- **map** `f list` : Apply `f` to each element of `list`.
- **mapAttrs** `f attrset` : Apply `f` to each attribute in `attrset`.
- **match** `regex str` : If `regex` matches `str`, return capture groups, else `null`.
- **mul** `e1 e2` : Multiply integers `e1 * e2`.
- **nixPath** : List of search path entries for lookups.
- **nixVersion** : String version of Nix.
- **null** : Literal `null`.
- **outputOf** `drv out` : Return output path of derivation.
- **parseDrvName** `s` : Parse a derivation name into components.

Bibliography